Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini
alex.santini2@studio.unibo.it

April 27, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for smart-city sensor data collection and replication, designed to operate across multiple nodes without any central coordinator. Each node may manage connections to multiple local sensors, ingest their readings through a pluggable sensor interface to support different sensor types, and maintain a local replica of the shared system state.

The platform disseminates sensor updates through push-pull gossip and resolves conflicts with deterministic timestamp-based Last-Writer-Wins merging, providing eventual consistency across replicas. In the demo setup, sensor streams are simulated for reproducibility, but the same ingestion model can be extended to real devices through dedicated adapters.

To remain operational in realistic distributed environments, the system combines gossip-based membership, phi-accrual failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while still guaranteeing convergence once communication is restored.

Fault-injection experiments on multi-node deployments show stable recovery and consistent replica alignment after failures and partitions, supporting the suitability of the approach for decentralized smart-city monitoring scenarios.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. The software is a distributed sensor-monitoring platform composed of autonomous peer-to-peer nodes and an optional web monitoring interface. More precisely, the project is primarily a distributed web-service platform: its main externally visible functionality is exposed through HTTP APIs served by executable, containerizable nodes. The optional HTML dashboard is only a lightweight observation client layered on top of the same APIs, while any compatible HTTP client can consume the service as well. Concretely, it combines:

- a set of autonomous nodes that collect and replicate sensor data without a central coordinator;
- an optional read-only web/introspection layer for monitoring and runtime observability.

Use case description. The system models a smart-city scenario in which multiple nodes ingest local sensor readings, maintain a local view of shared data, and exchange updates across the peer network. Each node can be deployed close to a local sensing point, connected to one or more sensor sources, and joined to the distributed network so that locally produced readings become part of a shared, replicated monitoring view.

The platform supports two main interaction profiles:

- *observational interaction*: users inspect sensor values, node health, and topology evolution;
- *operational interaction*: users configure nodes, connect sensor sources, attach nodes to the network, monitor health, and react to faults.

The system addresses three main usage profiles:

- *technical operators*, who administer the deployment by installing or starting node processes, configuring their network identity and bootstrap peers, connecting local sensor sources to the appropriate node, exposing the required service ports, monitoring health, and intervening when nodes fail or connectivity changes;

- *information operators and observers*, who inspect the replicated sensor state, membership information, topology knowledge, and operational views exposed by the system in order to understand what the distributed deployment is currently reporting;

- *automatic tools*, such as monitoring scripts, alerting components, integration services, or external dashboards, which consume the same read-only interfaces to exploit shared sensor and operational data without manually operating the distributed core.

Users interact with the system during deployment, routine supervision, and fault handling. Operators configure nodes and sensor connections when a sensing point is added or modified, start or restart nodes when the deployment changes, and check that the exposed APIs and peer connections are reachable. During normal operation, observers interact repeatedly through read-only HTTP endpoints such as `/api/state`, `/api/membership`, and `/api/introspection`, or through the dashboard, which polls the introspection API at regular intervals to display live state, membership, topology, events, and metrics. Additional interactions occur when failures, restarts, or network disruptions happen: operators inspect API responses, dashboard views, logs, metrics, and synchronization evidence to verify liveness transitions, recovery, and convergence. Thus, interaction is not continuous manual control of the distributed core, but periodic supervision and operational intervention around automated node-to-node communication.

The intended users are trusted operators and observers working on a development machine, Docker host, or controlled laboratory network. In the provided experimental setup, nodes run either as local Python processes or as Docker containers, while users access the exposed HTTP ports from the host machine through a browser, command-line HTTP clients, automated test harnesses, or the static dashboard. Therefore, the project is best understood as a reproducible distributed-systems experiment rather than as a public cloud service for external end users.

The system does not manage personal user data or application-level user records. Consequently, there is no need to store user data anywhere. The handled information is instead technical runtime data, such as sensor values, node metadata, membership state, metrics, and observability data.

Interaction channels and devices:

- HTTP API for programmatic interaction, monitoring, and integration with external tools;

- automatic monitoring, alerting, or integration tools consuming read-only API data;

- desktop/laptop web browser for optional dashboard-based monitoring.

Why distribution is needed. Distribution is a core requirement, not an implementation detail:

- sensing points are geographically and logically distributed;

- no single coordinator should be required for normal operation;
- communication delays, crashes, and temporary partitions must be tolerated;
- node-local views must eventually realign after faults are resolved.

The detailed behavioral requirements, protocol constraints, and acceptance criteria are specified in section 2.

2 Requirements Elicitation and Analysis

Requirements were elicited from the intended project goal described in section 1: build a decentralized laboratory platform for distributed sensor ingestion, replicated state propagation, fault observation, and monitoring. The elicitation process therefore focused on the observable services the platform must provide to operators and observers, on the quality attributes expected from a distributed-system prototype, and on the practical constraints imposed by course evaluation and reproducible experimentation.

2.1 Relevant Distributed System Features

Not all classical distributed-system features have the same relevance in this project. *Distributed Sensor Hub* is designed as a decentralized monitoring and experimentation platform, so the most important properties are decentralization, eventual convergence, fault observability, and reproducible multi-node execution. Other properties, such as strong security hardening or production-scale elasticity, are intentionally outside the main scope.

Distribution transparency. Only partially relevant. The system abstracts low-level TCP framing, message dispatching, local sensor ingestion, and merge logic from observers and dashboard clients. However, it does not try to hide the fact that the system is distributed. Replication delay, membership evolution, topology knowledge, phi-based liveness suspicion, anti-entropy exchanges, and recovery after failures are intentionally exposed through the read-only API and dashboard. In this project, observability is more important than full distribution transparency.

Decentralization. Highly relevant. Each node runs the full runtime stack independently: local sensors, state storage, membership management, failure detection, TCP protocol handling, push-pull replication, and HTTP observability. Normal operation does not rely on a master process, central coordinator, broker, or external database. Bootstrap peers are used only to seed communication; they are not permanent coordinators.

Fault tolerance and recoverability. Highly relevant. Nodes may crash, restart, or become temporarily unreachable during experiments. Surviving nodes are expected to keep ingesting local readings, serving read-only snapshots, exchanging updates with reachable peers, and maintaining their local membership view. A restarted or reconnected node recovers by exchanging missing deltas when possible and by falling back to full-state synchronization when incremental history is insufficient. The system prioritizes continued local availability and eventual recovery over uninterrupted global consistency.

The project does not require or guarantee a fixed recovery time. Fault detection and recovery are eventual and parameter-dependent: peer suspicion depends on heartbeat interval, Phi Accrual thresholds, accumulated heartbeat history, and network conditions, while state recovery depends on gossip cadence, pull frequency, delta-buffer retention, and the availability of at least one sufficiently up-to-date reachable peer. With the default one-second heartbeat interval, the phi thresholds provide an approximate suspicion window of a few seconds and a longer dead-peer classification window, but these are experimental expectations rather than hard real-time guarantees.

Failure detection and fault observability. Highly relevant. The membership subsystem uses heartbeat evidence and a Phi Accrual Failure Detector to derive peer states such as `alive`, `suspected`, and `dead`. These states are intentionally visible through membership and introspection endpoints, together with recent control-plane events and replication metrics. The goal is not only to tolerate failures, but also to make failure and recovery dynamics inspectable during distributed-systems experiments.

Consistency. Highly relevant. The replicated sensor state follows an eventual consistency model. Sensor updates are disseminated asynchronously through best-effort push-pull anti-entropy, and replicas may temporarily diverge during delays, crashes, or partitions. Convergence is obtained through a deterministic Last-Write-Wins policy based on `(ts_ms, origin)`, so nodes that receive the same set of updates eventually expose the same winning sensor records.

This means the project does not treat transient message loss or temporary replica divergence as unacceptable in the same sense as a strongly consistent production data service would. Instead, the intended guarantee is deterministic eventual convergence once reachable nodes exchange the same update set. Because the replicated state is volatile and maintained in memory, recovery after extended outages depends on at least one sufficiently up-to-date reachable peer, so permanent loss cannot be excluded if no such peer remains available.

Partition tolerance. Relevant. The project explicitly accepts degraded semantics during temporary communication partitions. Nodes in each connected component can continue local ingestion and observation, while replicated views may diverge. After communication is restored, gossip, delta exchange, and full-state synchronization allow reachable replicas to reconcile toward the same deterministic state.

Scalability. Relevant at course-project scale. The system is not intended for Internet-scale IoT deployments, but it must scale beyond a fixed two-node demonstration. The repository includes predefined Docker Compose topologies for small and medium experiments, including six-node and twelve-node scenarios. The main scalability objective is to preserve understandable convergence, membership behavior, and observability as the number of peers and sensor updates increases.

Resource sharing. Relevant. Nodes do not share centralized hardware resources; instead, they share distributed knowledge. This includes replicated sensor winners, peer membership information, liveness evidence, topology-related metadata, recent protocol events, and replication counters. Coordination is achieved through message exchange and deterministic merge rules rather than through a shared database or central service.

Openness, interoperability, and heterogeneity. Relevant. The implementation separates topology policy, sensor production, runtime processing, protocol messages, replicated state, and read-only observation clients. The topology layer is policy-driven, so the connection strategy can be injected or replaced without changing the rest of the runtime; in the current implementation, the supported production policy is the full-mesh policy. Sensor behavior is also modular: sensor providers are instantiated from configuration and deliver readings through a common ingestion boundary, allowing additional producers or adapters to be introduced while preserving the same downstream replication and observability pipeline. This makes the system open to heterogeneous sensor sources and alternative topology strategies, although the current project focuses on simulated local sensors rather than external industrial sensor protocols.

The system is also designed to interoperate with external components at clear boundaries. In particular, browser-based dashboards, generic HTTP clients, automated test scripts, and containerized deployment tooling interact with the same exposed observation surfaces, while node-to-node communication remains separated behind the internal TCP/JSON protocol.

Evolvability and maintainability. Highly relevant. The project is organized into explicit modules for runtime orchestration, networking, protocol handling, membership, failure detection, gossip, replicated state, sensors, topology, web API, and dashboard support. This separation is important because the platform demonstrates several distributed-systems concerns at once and must remain understandable enough to validate, debug, and extend.

Performance, concurrency, and communication efficiency. Relevant, but not subject to hard real-time constraints. Each node runs concurrent activities such as sensor production, state processing, heartbeat exchange, membership gossip, push-pull replication, TCP request handling, and HTTP polling. Communication efficiency matters because excessive pull traffic or insufficient delta retention can obscure convergence behavior. The implementation therefore exposes tuning parameters for gossip cadence, push and pull fanout, pull frequency, and replication-delta buffer size. The target is stable experimental behavior, not strict latency guarantees.

Accordingly, the project does not impose strict response-time or throughput requirements, nor does it define service-level guarantees. Timing behavior is intentionally parameter-dependent and should be understood as acceptable for small and medium course-project deployments rather than as a hard real-time commitment.

Security and trust. Limited in the current scope. The platform is designed for trusted local or laboratory environments, not adversarial multi-tenant deployments. It does not currently provide authentication, authorization, transport encryption, Byzantine fault tolerance, or protection against malicious peers. Security hardening would be a future extension rather than a core design driver of the present implementation.

This limited scope is also consistent with the nature of the handled information: the project does not process personal, sensitive, or confidential application data, but only simulated sensor readings and technical runtime metadata such as membership state, topology views, metrics, introspection events, and logs. Multiple conceptual roles exist, such as technical operators, observers, and automated tools, but the implementation does not enforce differentiated permissions because the exposed HTTP interfaces are read-only and unauthenticated.

Economy and cost. Relevant as an implementation constraint. The system is expected to run on ordinary development machines using repository-provided artifacts, Python dependencies, and Docker Compose topologies. It does not require paid cloud infrastructure, managed services, or institution-provided deployment platforms. This supports reproducibility, portability, and low-cost course evaluation.

Minimizing resource usage is therefore a secondary concern rather than a primary optimization target. Bounded buffers, configurable fanout, and limited experimental scale are useful to avoid unnecessary saturation, but the dominant objective remains understandable, reproducible distributed behavior on commodity hardware.

2.2 Glossary

The following terms are used throughout the report with the meanings adopted by this project:

Gossip Best-effort dissemination of node-local knowledge, especially membership and topology information, through periodic peer-to-peer messages.

Anti-entropy Periodic synchronization mechanism used to reduce divergence between replicas. In this project it is implemented as push-pull replication, where nodes both advertise local sensor deltas and request missing updates.

Phi Accrual Failure Detector Adaptive heartbeat-based failure detector that converts heartbeat inter-arrival timing into a suspicion score used to classify peers as `alive`, `suspected`, or `dead`.

Last-Writer-Wins Deterministic conflict-resolution policy in which the update with the greatest timestamp wins, with the origin identifier used as a deterministic tie-breaker when timestamps are equal.

Delta Incremental replication entry representing a sensor-state update together with sequence, origin, timestamp, value, and metadata. Deltas are retained in a bounded in-memory buffer.

Full sync Complete state transfer used when a node joins, rejoins, or can no longer recover from retained deltas alone.

Membership A node-local table of known peers, including network coordinates, liveness status, heartbeat evidence, phi score, and related metadata.

Bootstrap peer Initially configured peer endpoint used to seed discovery and start communication with the rest of the cluster.

Replicated sensor state The node-local Last-Writer-Wins view of the latest winning sensor records known by that node.

Introspection Read-only aggregate observation API exposing sensor state, membership, topology, events, and metrics using a stable schema.

Topology Node-local or merged knowledge about graph-like relations among peers, used mainly for observability and connection policy.

Eventual consistency Consistency model in which replicas may temporarily diverge, but converge after communication is restored and the same update set is exchanged.

Sensor provider Local producer that emits sensor readings into the ingestion pipeline.

Replication cursor Monotonic per-node sequence value used to request missing deltas during pull replication; it is not used for conflict resolution.

2.3 Functional Requirements

FR1 – Multi-node autonomous operation. The system shall allow multiple nodes to start independently and participate in the same distributed deployment without requiring a central coordinator.

Acceptance criterion. When a cluster of at least two configured nodes is started, each node becomes operational as an autonomous process and exposes its local service interfaces without depending on a single master node.

FR2 – Local sensor ingestion. Each node shall be able to manage one or more local sensor sources or sensor connections and ingest their readings into the distributed system state.

Acceptance criterion. Given at least one configured sensor on a node, the node eventually exposes fresh sensor records associated with its own origin through the read API or introspection API.

FR3 – Cluster-wide sensor-state dissemination. The system shall propagate sensor updates among nodes through decentralized replication so that readings produced on one node become visible on other reachable nodes.

Acceptance criterion. If a node generates new sensor readings while communication paths to other nodes are available, those readings eventually appear in the replicated sensor-state view exposed by the other reachable nodes.

FR4 – Deterministic timestamp-based conflict resolution. The system shall resolve concurrent or conflicting updates through a deterministic merge policy so that replicas can converge to the same winning value for each sensor record.

Acceptance criterion. When two or more updates compete for the same logical sensor record, all nodes that receive the same set of updates eventually expose the same winner for that record.

FR5 – Membership discovery and peer knowledge propagation. The system shall allow nodes to discover peers through bootstrap and peer-to-peer membership exchange, then maintain a distributed view of cluster membership.

Acceptance criterion. After startup and sufficient message exchange, a node exposes a membership view containing the peers it has discovered directly or indirectly.

FR6 – Adaptive liveness observation. The system shall provide operators with a node-local assessment of peer liveness so that missing or degraded peers can be observed during experiments.

Acceptance criterion. If a known peer becomes unreachable for long enough, at least one observing node eventually reports that peer as `suspected` or `dead` in its membership/introspection output.

FR7 – Recovery after crashes or temporary disconnections. The system shall allow a node that restarts or reconnects after temporary unavailability to recover the replicated state needed to rejoin normal operation by synchronizing with reachable peers.

Acceptance criterion. After a node crash or temporary isolation, once communication with at least one sufficiently up-to-date peer is restored, the recovering node eventually exposes a replicated state aligned with the rest of the reachable cluster.

FR8 – Read-only observability interface. The system shall provide a read-only programmatic interface for observing sensor state, membership, topology-related information, events, and operational metrics.

Acceptance criterion. A client can query documented HTTP `GET` endpoints and obtain structured snapshots of the node's current distributed-system view without modifying runtime state.

FR9 – Push-pull anti-entropy exchange. FR9 – Bidirectional replication synchronization. The replication layer shall support bidirectional synchronization between reachable peers so that each side can both advertise local updates and obtain missing information.

Acceptance criterion. During normal operation, reachable peers can send delta updates and request missing updates from one another, so each node can obtain sensor records it does not yet store.

FR10 – Phi Accrual failure suspicion output. FR10 – Liveness state exposure. The membership subsystem shall expose liveness states allowing operators to distinguish peers reported as `alive`, `suspected`, or `dead`.

Acceptance criterion. If liveness evidence from a peer degrades or stops for long enough, that peer is eventually reported as `suspected` or `dead` in membership/introspection output.

FR11 – Full-state rejoin synchronization. FR11 – Rejoin state reconstruction. When a node rejoins after crash, restart, or long partition, it shall support recovery of a coherent replica from reachable peers even when incremental synchronization alone is insufficient.

Acceptance criterion. After prolonged unavailability, a rejoining node can recover a sufficiently complete sensor-state view from a reachable peer and eventually align with the reachable cluster state, assuming such a peer still holds the relevant state.

FR12 – Human-oriented monitoring support. The system shall support human observation of the running cluster through a dashboard or equivalent visual monitoring interface.

Acceptance criterion. An observer can inspect live cluster information from a web browser through a dashboard that consumes the exposed read-only API of a node.

2.4 Non-Functional Requirements

NFR1 – Decentralization. Normal operation shall not depend on any central coordinator or single control component.

Acceptance criterion. The system continues to ingest local readings, maintain local state, and serve read-only observability data on surviving nodes even if one arbitrary peer becomes unavailable.

NFR2 – Eventual consistency. The replicated sensor state shall provide eventual, not immediate, consistency across nodes.

Acceptance criterion. In the absence of new conflicting updates and after communication is restored, reachable replicas converge to the same winning values for the same sensor records.

NFR3 – Partition tolerance with degraded semantics. The system shall remain available for local operation during temporary network partitions in experimental deployments, accepting that different partitions may temporarily diverge.

Acceptance criterion. During a partition, nodes in each connected component continue local ingestion and observation; after the partition heals, their replicas eventually converge again.

NFR4 – Fault observability. The system shall make failures and recovery visible enough to support distributed-systems experimentation and debugging.

Acceptance criterion. Operators can inspect status changes, replication activity, and recovery evidence through exposed membership, event, or metric views.

NFR5 – Reproducibility of experiments. The platform shall support repeatable validation scenarios on development machines and course evaluation environments.

Acceptance criterion. Using the documented setup and provided Docker Compose topologies, an operator can start a predefined cluster and reproduce representative convergence, partition, and recovery experiments without relying on external managed services.

NFR6 – Extensibility of sensor sources. The system shall be open to additional sensor producers without changing the conceptual role of the rest of the distributed runtime.

Acceptance criterion. New sensor sources can be introduced as additional producers while preserving the same downstream ingestion, replication, and observation capabilities.

NFR7 – Operational scalability at course-project scale. The platform shall support small and medium experimental clusters rather than only a fixed two-node demo.

Acceptance criterion. The documented deployment can be instantiated with multi-node topologies, such as six-node and twelve-node clusters, and still provides replicated state and observability functions for the whole experiment.

2.5 Implementation Constraints

IR1 – Self-contained local deployment. The project shall be executable on commodity development machines without requiring paid cloud services, institution-managed infrastructure, or external managed services.

Justification. The system is intended for course-project evaluation and distributed-systems experimentation. Its deployment model must therefore remain portable and reproducible from the repository itself.

Acceptance criterion. An operator can start one or more nodes using the repository code, documented dependencies, and provided deployment artifacts without provisioning external infrastructure.

IR2 – Reproducible predefined topologies. The project shall provide predefined multi-node deployment configurations that can be reused for demonstrations, validation, and fault-injection experiments.

Justification. Validation requires repeatable cluster scenarios with stable node identities, network coordinates, sensor configurations, and observation endpoints.

Acceptance criterion. An operator can launch the documented predefined topologies and obtain consistent node identities, sensor sources, peer relationships, and monitoring interfaces across repeated runs.

IR3 – Course-scale experimental scope. The implementation shall target small and medium experimental clusters suitable for local execution and course validation, rather than production-scale IoT deployments.

Justification. The system is designed to demonstrate distributed-systems properties under controlled conditions, not to satisfy industrial-scale availability, performance, or device-management requirements.

Acceptance criterion. The provided configurations and validation scenarios support representative multi-node deployments suitable for evaluating convergence, partition behavior, recovery, and observability at course-project scale.

3 Design

This chapter describes the high-level design adopted to satisfy the requirements in section 2. The main design goal is to support autonomous nodes (FR1), local sensor ingestion and replicated convergence (FR2–FR4), decentralized membership and liveness observation (FR5–FR6), recovery after failures (FR7), and read-only monitoring (FR8, FR12), while preserving decentralization, eventual consistency, partition tolerance, and course-scale experimental scalability (NFR1–NFR3, NFR7).

3.1 Architecture

The system follows a **fully decentralized peer-to-peer architecture** in which every node executes the same logical stack and can both produce local sensor data and consume data generated by other peers. There is no master node, no dedicated coordinator, and no centralized database. This architectural style directly addresses FR1 and NFR1: any node can participate in the cluster as an autonomous peer, and the failure of one peer does not invalidate the whole system.

From a logical viewpoint, the architecture is organized into three cooperating planes:

- a **control plane**, responsible for decentralized peer discovery, membership maintenance, peer knowledge propagation, adaptive liveness observation, Phi Accrual status classification, and topology knowledge;
- a **data plane**, responsible for local sensor ingestion, local state maintenance, deterministic merge semantics, push-pull anti-entropy exchange, and full-state rejoin synchronization;
- an **observability plane**, responsible for exposing cluster snapshots, fault evidence, metrics, and monitoring views to operators and observer clients.

This separation is important because the project intentionally distinguishes between *knowing who is in the system* and *replicating what the system knows about sensors*. Membership and liveness information evolve under different timing assumptions and serve different purposes than sensor-state replication. Keeping the two concerns separate improves modularity, clarity, and evolvability, which supports NFR6 and the maintainability goals discussed in section 2.1.

Within each node, the architecture is further layered:

- a **transport layer**, dealing only with network communication;
- a **protocol layer**, defining message types and dispatching received messages to the correct logic;
- a set of **domain components**, where the actual semantics of membership, failure detection, replication, topology, and observation are applied.

This layered organization keeps low-level communication concerns separate from distributed-system semantics. As a consequence, the same conceptual design would still hold if the underlying transport or encoding technology changed, which is consistent with the intended role of the design chapter.

3.2 Infrastructure

The infrastructure is intentionally minimal. The essential infrastructural unit is the **node process**, replicated one time for each participant in the cluster. Every node contains the same internal subsystems: sensor ingestion, local state management, peer membership handling, failure detection, TCP protocol handling, and a read-only HTTP observation interface. This homogeneous-node model simplifies deployment and supports both FR1 and NFR5, because the same runtime can be instantiated in different topologies without role specialization.

No external infrastructural component is required for normal operation. In particular, the design does not rely on:

- a central database;
- a message broker;
- a coordination service;
- a load balancer;
- a dedicated monitoring backend.

The only optional external-facing component is the browser dashboard, which is not part of the distributed core and behaves as a read-only observer. It can be hosted separately because it only consumes the observation API exposed by any node.

The distributed deployment model is therefore the following:

- one or more nodes can run as separate processes, containers, or machines, while distributed behavior is exercised when at least two nodes are active;
- each node may manage a different local set of sensor sources or sensor providers;
- nodes communicate directly with peers over the inter-node network;
- observers may connect to any node's read-only HTTP API, or use the dashboard configured to consume that API.

Component discovery is bootstrapped through a configured set of initial peers. After first contact, discovery is no longer purely static: nodes can exchange peer knowledge and extend their local membership view through join messages, peer-list exchange, membership gossip, and full-state synchronization metadata. Naming is based on stable node identifiers plus network coordinates required for communication. This choice is sufficient for the project scope because it avoids introducing centralized service-discovery infrastructure while still allowing evolving cluster knowledge, which is coherent with FR5 and NFR1.

From a physical standpoint, the design does not assume geographic distribution at continental scale; however, it does assume that nodes may be distributed across separate machines or containers and may suffer message delay, temporary unreachability, or partial connectivity. This is exactly the failure model needed to study NFR2 and NFR3.

3.3 Modelling

The main domain entities are:

- **Node**, representing an autonomous distributed participant;
- **Peer**, representing another known node together with its membership and liveness metadata;
- **Sensor source**, representing a local producer of readings;
- **Sensor reading**, representing one observation generated by a sensor source;
- **Replicated sensor record**, representing the current winning value known for one logical sensor identifier;
- **Membership entry**, representing a node's local belief about another peer's status;
- **Topology entry**, representing adjacency knowledge useful for network observability and connection policies;
- **Observer view**, representing an aggregated, read-only projection of internal distributed state.

These entities map naturally to infrastructural components. Sensor sources are local to a single node, while replicated sensor records, membership entries, and topology entries are stored as node-local replicas. In other words, the design does not centralize domain state in one authoritative repository; instead, each node stores its own best-effort replica of the distributed knowledge relevant to operation and observation.

The most important domain events are:

- production of a local sensor reading;
- discovery of a new peer;
- reception of a heartbeat;
- transition of a peer between liveness states;
- emission or reception of a sensor-state update;

- request for missing replication data;
- full-state transfer during recovery;
- generation of an introspection snapshot for observers.

The exchanged messages can be grouped into three categories:

- **control messages**, for discovery, membership dissemination, and liveness observation;
- **state-replication messages**, for sensor update propagation and catch-up;
- **read-only queries**, for HTTP observation by users or tools.

The system state includes:

- latest winning sensor values together with origin and timestamp metadata;
- membership knowledge about known peers;
- liveness evidence and peer-status information;
- topology knowledge and connection-related views;
- operational events and metrics used for observability.

This model is intentionally centered on *current replicated knowledge*, not on long-term historical storage. That design keeps the project focused on convergence, liveness, and fault handling rather than on archival analytics.

3.4 Interaction

There are two main interaction families in the system.

The first is **peer-to-peer interaction among nodes**. This interaction is best-effort, message-based, and decoupled through protocol dispatch. It is used for:

- bootstrap and peer discovery;
- heartbeat exchange;
- dissemination of membership knowledge;
- dissemination and retrieval of replicated sensor state;
- recovery synchronization after missed updates.

The second is **observer interaction**, where external clients query a node through the HTTP read API. This interaction is request-response and read-only. It does not participate in cluster coordination and it never mutates the distributed state.

At the inter-node level, communication follows a layered path:

domain intent → protocol message → transport delivery → protocol dispatch → domain update

The main interaction patterns are:

- **bootstrap and peer-list exchange**, used when a node joins and needs initial peer knowledge;
- **periodic heartbeat exchange**, used to collect liveness evidence;
- **gossip dissemination**, used to spread membership knowledge without central coordination;
- **push-pull state synchronization**, used to exchange recent sensor-state updates efficiently;
- **full synchronization on demand**, used when incremental recovery is insufficient;
- **polling-based observation**, used by the dashboard and external tools to inspect cluster state.

This design supports FR3, FR5, FR6, FR7, FR8, FR9, FR10, FR11, and FR12 because it combines low-coupling peer communication with an observation interface that stays orthogonal to the replicated core.

3.5 Behaviour

The behavior of the main components can be summarized as follows.

Sensor sources are active components: they periodically generate local readings and inject them into the node. They are conceptually producers and do not need global knowledge.

The local state manager is stateful and authoritative for the node's replica. It receives local and remote updates, applies deterministic merge rules, stores the current winners, and exposes snapshots to both replication and observability components. This component is central to FR2, FR3, and FR4.

The membership manager is also stateful. It stores known peers, updates direct or indirect liveness evidence, and maintains the node's local view of cluster membership.

The failure detector behaves as a local evaluator rather than as a replicated authority. It interprets heartbeat timing and produces suspicion levels for peers observed directly by the node. Those local judgements are then translated into membership information that can be disseminated.

The heartbeat and gossip runtime is periodic. At each round, it evaluates liveness, sends heartbeat probes, and disseminates updated membership knowledge. Its purpose is not to guarantee exact membership synchronization at every instant, but to make peer-status knowledge progressively converge.

The replication runtime is periodic as well. It pushes recent sensor-state updates to selected peers and pulls missing updates from others. When incremental recovery is not sufficient, it can trigger a broader synchronization phase. This behavior is essential for FR7 and NFR2.

The observability subsystem is read-only. It collects snapshots from state, membership, topology, and event/metric providers, then publishes a coherent observation view for users and tools.

State updates are therefore concentrated in a small set of stateful components:

- local sensor-state storage;
- local membership storage;
- local topology and observability stores.

Other components mainly trigger transitions or move data between these stores. This separation keeps write paths explicit and supports maintainability.

3.6 Data and Consistency Issues

The system stores several forms of node-local runtime data:

- the replicated sensor-state winners;
- the incremental replication view needed for propagation and recovery;
- the membership table and liveness metadata;
- topology knowledge;
- recent events and metrics for introspection.

This data is primarily **runtime state**, not long-term persistent storage. The design does not require an external database because the goal is to study distributed replication and convergence of current system knowledge rather than durable business records. This keeps the architecture lightweight and self-contained, matching IR1.

Conceptually, the replicated sensor state behaves like a key-value map from a logical sensor identifier to the current winning record. The important property is not relational querying power, but deterministic merge semantics. For this reason, the core consistency problem is solved at the application level rather than delegated to a storage engine.

Consistency is handled differently for different data types:

- **sensor state** uses eventual consistency with deterministic Last-Writer-Wins conflict resolution based on timestamp and origin metadata;
- **membership state** also converges eventually, but it reflects local observations that may temporarily differ across nodes;
- **observability data** is inherently approximate and time-sensitive, because it represents a snapshot of changing distributed state.

This means the system explicitly rejects strong consistency as a primary goal. During partitions or transient delays, different nodes may expose different but still valid local views. Once communication resumes and no newer conflicting updates appear, the replicas converge. This choice is aligned with NFR2 and NFR3.

Data is shared among components in well-defined ways:

- sensor producers share generated readings with the local state manager;
- the state manager shares snapshots and deltas with the replication runtime and the observability subsystem;
- the membership subsystem shares peer-status information with heartbeat, gossip, topology, and observability components;
- the observability subsystem reads from other stores but does not modify them.

This read/write asymmetry is deliberate: observation must remain decoupled from mutation so that FR8 can be satisfied without introducing accidental feedback paths.

Concurrent access to local state is expected because sensor ingestion, TCP protocol handlers, replication publishing, heartbeat evaluation, and HTTP polling may run at the same time. The design therefore separates write ownership and read access. Local sensor producers do not conceptually mutate the replicated state directly: they emit readings into an ingestion path consumed by the local state manager. Remote updates and full-sync responses also enter through explicit merge operations, while observers receive snapshots rather than direct references to mutable stores.

The consequence is that concurrent reads and writes are allowed, but they are confined to node-local stores and protected by implementation-level synchronization. There is no distributed transaction spanning sensor state, membership, topology, and observability data. This limitation is acceptable because these views are intended to be eventually consistent, approximate, and inspectable rather than globally atomic.

3.7 Fault-Tolerance

Fault tolerance is one of the central design drivers of the project. The first mechanism is **state replication**: sensor-state knowledge is disseminated among peers so that the system does not depend on a unique owner of cluster knowledge. Replication is best-effort and decentralized, which supports NFR1 and NFR3 even though it does not provide instantaneous consistency.

The second mechanism is **heartbeat-based liveness observation**. Nodes periodically exchange liveness probes and evaluate the timing of received heartbeats. This allows each node to form a local judgement about whether a peer is alive, suspected, or dead. Since such knowledge is local and timing-dependent, the design treats liveness as a continuously revised estimate rather than as an absolute truth.

The third mechanism is **membership gossip**. A node does not keep liveness knowledge only for itself; it disseminates updated membership information so that peer knowledge can propagate even when direct connectivity patterns are incomplete. This improves resilience of membership awareness and helps the system maintain a broader view of the cluster.

The fourth mechanism is **anti-entropy recovery**. Incremental propagation is efficient when peers stay mostly synchronized, but it may be insufficient after outages, restart, or missed traffic. For this reason, the design includes a stronger recovery path that allows a lagging node to regain a coherent replicated view after disruption, directly supporting FR7.

Error handling is intentionally failure-aware rather than failure-oblivious. If a component or peer becomes temporarily unavailable:

- surviving nodes continue local operation;
- local replicas may diverge temporarily;
- membership views reflect degraded reachability;
- synchronization resumes when communication becomes possible again.

This behavior is preferable to halting the whole system because the project prioritizes availability and observation under faults over strong global coordination.

3.8 Availability

The design does not introduce classical web-style caching layers or external load balancers, because the system is not centered on high-volume client traffic toward a centralized service. Instead, availability is achieved structurally through replication, autonomy of nodes, and local read access to each node's current view.

Each node acts as a locally available observation point. As long as a node remains running, it can still:

- ingest its own local sensor readings;
- maintain its local replica;
- expose read-only snapshots to operators and tools.

In this sense, replicated state itself plays a role analogous to availability-oriented caching: each node keeps a local working copy of shared cluster knowledge, so read operations do not require contacting a central authority.

No explicit load balancing is required because there is no single mandatory ingress point for observation. Different observers may query different nodes, and each node exposes its own local view. This is sufficient for the experimental scope of the project.

During a **network partition**, the design favors continued local operation over blocking. Nodes inside each connected component continue to exchange information among themselves, while nodes separated by the partition stop receiving fresh updates from one another. Consequently:

- sensor-state replicas may diverge temporarily across partitions;

- membership views may mark remote peers as suspected or dead;
- local observation remains available inside each connected component.

When the partition heals, incremental dissemination and recovery mechanisms progressively re-align the replicas. This behavior is a direct consequence of choosing availability and partition tolerance over strong consistency, and it is exactly the semantics required by NFR2 and NFR3.

3.9 Security

Security is intentionally limited in the current design because, as discussed in section 2.1, it is not a primary driver of this project. The system is assumed to run in controlled laboratory, development, or trusted demonstration environments.

Accordingly, the current design does **not** make authentication a core architectural dependency for either peer-to-peer traffic or read-only observation endpoints. Similarly, it does not define a rich authorization model with differentiated user permissions. The effective roles are simple: nodes participate in the distributed protocol, while observers query read-only information.

This choice keeps the architecture aligned with the educational scope of the project and avoids obscuring the main distributed-systems concerns with security infrastructure that is orthogonal to the report goals. However, the design has been kept modular enough that stronger security policies could be introduced later at clear boundaries:

- on node-to-node channels, to authenticate peers and protect inter-node communication;
- on HTTP observation endpoints, to restrict access to monitoring data;
- on deployment boundaries, through network isolation and reverse-proxy policies.

Likewise, no cryptographic protocol is a conceptual prerequisite of the current system model. Confidentiality and strong identity guarantees are therefore outside the present scope and remain future extensions rather than core design commitments.

4 Implementation

This chapter reports the concrete technology-dependent choices used to realize the design discussed in section 3. The implementation deliberately favors a lightweight and inspectable stack, so that the distributed behavior of the system remains visible and reproducible without relying on heavy external infrastructure.

4.1 Protocols and Data Representation

Two communication protocols are used, each chosen according to the interaction style required by the system.

Node-to-node communication. Peer communication is implemented over **TCP**. This choice was preferred over UDP because the project needs ordered and reliable delivery of protocol payloads at the transport level, while still keeping the communication stack simple and under direct control. TCP is therefore used for:

- bootstrap and peer discovery traffic;
- heartbeat exchange;
- membership gossip dissemination;
- sensor-state update propagation;
- incremental and full synchronization during recovery.

Rather than adopting an external RPC or messaging framework, the project uses a custom lightweight application protocol on top of TCP. Payloads are transmitted as length-prefixed frames, which makes message boundaries explicit and keeps the implementation compatible with a persistent stream-oriented transport.

Observer communication. Human users and external tools interact with the system through a **read-only HTTP API**. HTTP is appropriate here because the observability plane is request-response oriented, browser-friendly, and naturally suited to dashboards, integration scripts, and test harnesses. Since the observation API does not participate in peer coordination, it is intentionally separated from the internal peer-to-peer protocol.

In-transit representation. Messages and HTTP responses are represented as **JSON**. This choice was motivated by readability, ease of debugging, and direct interoperability with browsers and Python tooling. JSON is sufficient for the project because payload sizes are moderate and the main goal is clarity rather than binary compactness. Message validation and dispatch are then handled at the protocol layer after decoding.

The main message families implemented in the peer protocol are:

- `JOIN_REQUEST` and `PEER_LIST` for discovery;
- `PING` and `PONG` for heartbeat-based liveness observation;
- `GOSSIP_STATE` for disseminating membership knowledge;

- `SENSOR_UPDATE` for push-based sensor replication;
- `GET_DELTA` and `DELTA_UNAVAILABLE` for incremental recovery;
- `FULL_SYNC_REQUEST` and `FULL_SYNC_RESPONSE` for stronger catch-up after disruption.

4.2 State Management and Storage Choices

The implementation does **not** use an external database. All relevant information is maintained as in-memory runtime state inside each node:

- the current winning sensor records;
- replication deltas and sequence metadata;
- the peer membership table;
- liveness-related information;
- topology snapshots;
- introspection events and counters.

This is consistent with the project scope: the goal is to study replication, convergence, recovery, and observability of current distributed state, not durable long-term business storage. As a consequence, no SQL or NoSQL query layer is required. Queries are replaced by direct access to in-memory state providers, which are then exposed through the HTTP observation endpoints.

From an implementation viewpoint, the most important storage choice is therefore not the selection of a database technology, but the decision to encode merge semantics explicitly in the application logic. Sensor-state convergence is implemented with deterministic Last-Writer-Wins conflict resolution based on timestamp and origin metadata, while membership and liveness views are maintained through dedicated in-memory structures updated by protocol handlers and periodic runtimes.

4.3 Authentication and Authorization

No explicit authentication or authorization protocol is implemented in the current version of the system. This is a deliberate scope decision, not an omission by accident. The project runs in controlled development and laboratory environments, and the report prioritizes decentralization, consistency, failure handling, and observability over access-control concerns.

Accordingly:

- peer-to-peer channels do not currently require node authentication;
- the HTTP API does not require login, tokens, or session management;

- the system does not implement role-based or attribute-based authorization.

The practical effect is that any process able to reach a node in the current deployment can interact with the exposed interface allowed by that channel. This is acceptable for the project scope, but it also marks a clear limitation of the current implementation. Stronger authentication, endpoint protection, and transport hardening are left as future work rather than embedded in the present runtime.

4.4 Technological details

Programming language and runtime. The project is implemented in **Python**. This choice supports rapid experimentation, readable code, and low setup cost for a course project. The implementation makes extensive use of the Python standard library for networking, concurrency, HTTP serving, serialization, logging, and timing, which keeps external dependencies intentionally minimal.

Concurrency model. The runtime is based on **threads**, background worker loops, and in-memory queues. This model is used for several long-lived activities that must proceed concurrently:

- accepting inbound TCP connections;
- maintaining outbound peer workers;
- periodic heartbeat and failure-detector evaluation;
- periodic push-pull replication;
- sensor generation;
- HTTP serving for observation.

This approach was preferred to a more complex actor framework or asynchronous event-loop architecture because it keeps the execution model explicit and understandable, which is desirable for an educational distributed-systems project.

Concurrent access to local replicated state is handled through a combination of queue-based ingestion and explicit locking. Local sensor providers enqueue normalized sensor events into a thread-safe event queue consumed by the `NodeStateWorker`; remote protocol handlers can invoke merge operations for `SENSOR_UPDATE` and `FULL_SYNC_RESPONSE` messages. The underlying `NodeStateStore` protects sensor records, update buffers, replication deltas, sequence cursors, and origin-sequence tracking with a mutex. Membership and failure-detector state are protected separately by locks in the peer table and heartbeat monitor. This mitigates in-process race conditions among sensor generation, TCP handlers, replication publishing, heartbeat evaluation, and HTTP polling. The remaining limits are deliberate: state and membership are not updated atomically as one transaction, replication buffers are bounded and volatile, and recovery relies on peer synchronization rather than durable local storage.

Configuration management. Node configuration is externalized through **environment variables**. Parameters such as node identity, ports, bootstrap peers, sensor definitions, replication cadence, failure-detector thresholds, and fault-injection options are all configurable from the environment. This makes it easy to instantiate different topologies and experimental scenarios without changing source code.

Sensor implementation. Sensor sources are implemented as pluggable local producers. The repository includes multiple simulated sensor types, such as numeric, boolean, wave, noise, spike, trend, categorical, incremental, and finite test sensors. This confirms the extensibility objective identified in NFR6 while keeping the whole platform reproducible in the absence of real hardware devices.

Networking layer. The transport layer is implemented with Python sockets and a custom framed TCP protocol. On the outbound side, each peer connection has its own worker and FIFO send queue; on the inbound side, a threaded TCP server accepts connections, reads framed messages, validates them, and dispatches them to protocol handlers. The implementation also includes bounded worker counts and configurable socket limits, which helps prevent unbounded resource growth in larger experiments.

HTTP observation layer. The Web API is implemented with Python's 'http.server' facilities, in particular a threaded HTTP server and custom request handlers. This is sufficient because the API is read-only and relatively small, so introducing a larger web framework would not provide proportionate benefits for the project scope.

Containerization and deployment automation. The project uses **Docker** and **Docker Compose** to instantiate reproducible node topologies. Multiple compose configurations are provided, including small and medium cluster layouts, so that the same codebase can be exercised under different scales and fault scenarios. Containerization strongly supports NFR5 because it reduces environmental variability across test runs and demonstrations.

Testing and quality tools. Validation and development are supported by:

- **pytest** for unit and integration testing;
- Docker-backed integration harnesses for multi-node and fault-recovery scenarios;
- **pyright** for static type checking;
- structured logging and introspection endpoints for runtime diagnosis.

Dependencies. External dependencies are intentionally few. Besides development tools, the main runtime dependency is 'python-dotenv', used to simplify environment-based configuration. This minimal dependency footprint is coherent with the project's emphasis on portability, inspectability, and self-contained deployment.

4.5 Comparison with Messaging and Streaming Frameworks

One of the learning goals of the project is to compare the implemented peer-to-peer approach with established distributed messaging and streaming technologies. The system intentionally avoids adopting these frameworks as core dependencies because the educational objective is to expose the mechanisms of gossip dissemination, membership maintenance, failure suspicion, and recovery rather than delegating them to middleware.

MQTT. MQTT is well suited to IoT telemetry because it offers a lightweight publish/subscribe abstraction and a broker-centered communication model. It would simplify sensor ingestion and client distribution, but the broker would become an explicit infrastructural dependency and, unless clustered externally, a possible coordination point. In comparison, Distributed Sensor Hub keeps dissemination decentralized: every node participates directly in replication and membership, which makes failures and convergence behavior more visible for experimentation. The trade-off is that this project must implement discovery, peer communication, and recovery logic itself, while MQTT would provide a simpler application-facing interface.

ZeroMQ. ZeroMQ provides flexible messaging patterns and efficient socket abstractions without imposing a broker in all configurations. It could reduce the amount of low-level networking code and support several communication topologies. However, it does not by itself define a replicated sensor-state model, membership protocol, Phi Accrual failure detector, or full-state rejoin semantics. Using ZeroMQ would therefore help with transport engineering but would not replace the distributed-system logic that this project wants to study directly. The custom TCP and JSON protocol is less general, but keeps the behavior inspectable and aligned with the course goals.

Kafka. Kafka provides durable logs, high-throughput ingestion, replay, and strong operational support for stream processing. It is much stronger than this prototype for production-scale persistence, ordering within partitions, and consumer coordination. At the same time, Kafka relies on a managed broker cluster and a log-centric data model. That would shift the project away from autonomous peer replicas toward a centralized streaming substrate. Distributed Sensor Hub instead sacrifices durable history and large-scale throughput in favor of node autonomy, local availability during partitions, and direct observation of eventual consistency through timestamp-based merging.

Redis Streams. Redis Streams offers a compact and practical stream abstraction with consumer groups and persistence options. It would be useful for buffering sensor events and coordinating consumers in a small deployment, but it still introduces a server-side data store as a central component unless Redis is deployed and managed in a replicated configuration. The implemented system does not need an external store for normal operation: each node keeps its own runtime replica and repairs it through gossip, deltas, and full synchronization. This improves conceptual decentralization, while Redis Streams would provide stronger operational convenience and better persistence primitives.

Overall, the implemented architecture is not meant to outperform these frameworks in production maturity. Its value is educational and experimental: it makes consistency, scalability, and fault-tolerance trade-offs explicit. MQTT and Redis Streams would simplify integration but introduce broker or server dependencies; ZeroMQ would improve transport flexibility but leave higher-level replication semantics to the application; Kafka would provide durable scalable streaming but change the architecture into a brokered log-based system. Distributed Sensor Hub instead prioritizes decentralized membership, anti-entropy replication, adaptive failure detection, and recovery after partitions, accepting simpler storage and course-project scale as deliberate constraints.

5 Validation

Project validation was designed across multiple layers to verify both the functional correctness of individual modules and the emergent behavior of the distributed system under replication, failures, and reconnection. The adopted strategy combines unit tests, integration tests, smoke tests in a containerized environment, and manual observation scenarios, with the goal of covering the entire node lifecycle: startup, bootstrap, state propagation, failure detection, recovery, and introspection.

5.1 Automatic Testing

Unit tests. The foundation of the validation plan is a unit test suite developed with `pytest`. Unit tests verify, in isolation, the responsibilities of the system's main modules:

- `tests/protocol/`: correctness of serialization, deserialization, validation, and dispatch of application-protocol messages;
- `tests/networking/`: TCP framing, client/server connection handling, operational limits, and transport robustness;
- `tests/state/`: deterministic application of the Last-Writer-Wins policy and maintenance of incremental deltas;
- `tests/membership/` and `tests/fd/`: membership-view updates, heartbeat handling, and *phi-accrual* estimation for peer classification;
- `tests/sensors/`: behavior of simulated sensor generators and the related lifecycle manager;
- `tests/webapi/` and `tests/introspection/`: consistency of read-only HTTP endpoints and of the introspection contract exposed to clients.

The rationale for this suite is twofold: on the one hand, to prevent regressions in components with local semantics; on the other hand, to make explicit the invariants that the system must preserve regardless of distributed topology. At the time of local repository verification, running `python -m pytest -m "not integration" -q` produced `180 passed, 9 deselected`, confirming the stability of the non-integrated portion of the suite.

In-process integration tests. A second validation layer exercises interaction among multiple real nodes executed within the same test process or in processes coordinated by dedicated harnesses. In particular:

- `tests/integration/test_deterministic_state_convergence.py` verifies replicated-state convergence in a six-node cluster, checking that final LWW winners match expected outcomes;
- `tests/integration/test_cluster_harness.py` validates the correct behavior of the supporting infrastructure for reproducible distributed scenarios;

- `tests/integration/test_finite_test_sensor.py` uses deterministic sources to make update sequences observable and comparable across nodes.

These tests cover properties that cannot be inferred from local correctness alone: incremental propagation, relative ordering of updates, initial bootstrap, and eventual convergence after multiple message exchanges.

Docker-based integration tests and production-like scenarios. The third validation layer uses Docker Compose topologies to exercise the software in an environment closer to real deployment. This category includes:

- `tests/integration/test_docker_deterministic_state_convergence.py`, which replicates convergence verification on containerized nodes;
- `tests/integration/test_docker_crash_restart_recovery.py`, which evaluates recovery after node stop and restart;
- `tests/integration/test_docker_partition_reconciliation.py`, which verifies state reconciliation after a connectivity discontinuity;
- `tests/integration/verify_cluster.py`, used as a smoke test to check correct cluster startup and reachability of primary endpoints.

The main advantage of this solution is that the same deployment setup used for demonstrations is reused as test infrastructure. This reduces the distance between development, verification, and presentation environments, improving the reproducibility of experiments.

Execution automation. Automated test execution is also supported by CI workflows defined in `.github/workflows/unit-tests.yml` and `.github/workflows/integration-tests.yml`. The unit-test pipeline installs dependencies and runs `pytest` while excluding integration tests, whereas the integration pipeline runs both deterministic convergence checks and a smoke test on Docker clusters. This automation helps detect regressions in both local logic and the most relevant distributed scenarios.

How to run the tests. The main execution modes, already documented in the repository, are the following:

- unit and modular tests: `pytest --maxfail=1` or `python -m pytest -v -m "not integration"`;
- protocol tests: `pytest -m protocol`;
- targeted verification of LWW convergence: `python -m pytest tests/integration/test_determini` `-q -s`;
- Docker smoke test: start the Compose topology, then run `python tests/integration/verify_clust` `--timeout 60 --interval 2`.

Coverage with respect to project goals. Overall, automated tests cover the most important points identified in the project requirements:

- correctness of the communication protocol and its validations;
- deterministic convergence of replicated state;
- propagation of membership and liveness information;
- tolerance to crashes, restarts, and temporary partitions;
- external verifiability through HTTP APIs and introspection.

More explicitly, the main test groups map to requirements as follows:

`tests/state/test_lww.py` verifies Last-Writer-Wins merge behavior, stale-update rejection, tie breaking, full-state merge, partition convergence, delta buffers, and cursors. It covers FR3, FR4, FR7, FR9, FR11, NFR2, and NFR3.

`tests/fd/` **and** `tests/membership/` verify phi estimation, peer-table updates, heartbeat transitions, suspected/dead/alive recovery, gossip status merging, and membership snapshots. They cover FR5, FR6, FR10, and NFR4.

`tests/protocol/test_full_sync_handlers.py` verifies `GET_DELTA`, `DELTA_UNAVAILABLE`, full-sync requests, full-sync responses, and recovery fallback. It covers FR7, FR9, FR11, and NFR2.

`tests/protocol/test_gossip_state_handlers.py` verifies membership gossip, stale status rejection, delayed convergence after partitions, and topology merge. It covers FR5, FR6, NFR1, NFR3, and NFR4.

`tests/runtime/test_sensor_update_publisher.py` verifies push-pull fanout, alive-peer targeting, and pull tracking. It covers FR3, FR9, NFR2, and NFR7.

`tests/webapi/` **and** `tests/introspection/` verify read-only HTTP endpoints and aggregate introspection snapshots. They cover FR8, FR10, FR12, and NFR4.

`tests/sensors/` verifies simulated sensor providers and sensor-manager configuration. It covers FR2 and NFR6.

`tests/networking/` verifies TCP framing, connection handling, concurrency, slow clients, bursts, limits, and shutdown. It covers FR1, FR3, NFR1, and NFR7.

`tests/topology/` verifies topology-state merge and full-mesh policy resolution. It covers FR5, FR8, NFR7, and IR3.

`tests/integration/test_deterministic_state_convergence.py` verifies that six real in-process nodes converge to deterministic expected winners. It covers FR1, FR2, FR3, FR4, FR9, NFR1, NFR2, NFR7, and IR2.

`tests/integration/test_cluster_harness.py` verifies that the reusable cluster harness starts nodes and exposes state and membership. It covers FR1, FR5, FR8, IR1, and IR2.

`tests/integration/test_docker_deterministic_state_convergence.py` verifies convergence in container networking. It covers FR1, FR2, FR3, FR4, FR9, NFR2, NFR5, NFR7, IR1, and IR2.

`tests/integration/test_docker_crash_restart_recovery.py` verifies crash/restart recovery, state reconciliation, and membership recovery. It covers FR6, FR7, FR10, FR11, NFR3, NFR4, and NFR5.

`tests/integration/test_docker_partition_reconciliation.py` verifies temporary network partition, divergence, healing, and reconciliation. It covers FR3, FR4, FR7, FR9, FR11, NFR2, and NFR3.

`tests/integration/verify_cluster.py` performs a Docker smoke check that each node exposes replicated data from expected origins. It covers FR1, FR3, FR8, NFR5, IR1, and IR2.

The rationale of the main distributed tests is also requirement-driven. Deterministic convergence tests prove that autonomous nodes exchange finite sensor streams and converge to the same Last-Writer-Wins result, intercepting silent replication failures, incorrect fanout, or non-deterministic merge behavior. Docker convergence repeats the same idea in container networking, where DNS names, exposed ports, startup timing, and network isolation are closer to deployment conditions. Crash/restart recovery validates that a restarted node, after losing volatile in-memory state, can catch up through delta exchange or full synchronization, while surviving peers continue operating. Partition reconciliation checks that the system can remain locally available during a split, accept temporary divergence, and converge again after communication is restored. Full-sync handler tests specifically protect against the risk that stale cursors make recovery impossible, while web API and introspection tests protect the dashboard and external tools from unstable or incomplete observation contracts.

5.2 Acceptance test

Alongside automated validation, manual acceptance tests were also executed, primarily focused on the system's observable and experimental aspects. These tests were performed through:

- startup of 2-node, 6-node, and 12-node Docker clusters;
- direct inspection of `/api/state`, `/api/membership`, `/api/introspection`, and related derived views;
- use of the static dashboard in `web/` to observe topology, recent events, replication metrics, and peer status;
- execution of `manual_tests/compose_chaos.py` to selectively stop and restart services during cluster execution.

These manual tests were intended to confirm properties that are difficult to capture fully through automated assertions alone: readability of exported state, quality of observability, temporal consistency of dashboards, and perceivable system behavior during reconfiguration and network transients.

The manual component does not replace the automated suite, but complements it. In particular, it proved useful when acceptance criteria were also qualitative, for instance in evaluating the clarity of information shown to users or in visually diagnosing phenomena such as pull storms, accumulated delays, or rapid membership state transitions.

6 Deployment

Project deployment was designed with two complementary objectives: enabling rapid execution on a single machine for development and debugging activities, while ensuring reproducibility of multi-node topologies through containerization. System distribution does not require external infrastructure, dedicated orchestrators, or persistent supporting services: each node autonomously executes its own networking logic, replication, failure detection, and HTTP observability.

6.1 Software prerequisites

To run the project from scratch, the following tools are required:

- a recent version of **Python 3**; the repository CI pipeline explicitly validates execution with `Python 3.12`;
- **pip** for Python dependency installation;
- **Docker** and **Docker Compose** for launching containerized topologies;
- optionally, a web browser to inspect the observability dashboard and HTTP endpoints exposed by nodes.

Runtime dependencies are intentionally minimal and defined in `requirements.txt`; development and static-analysis tools are instead collected in `requirements-dev.txt` and in `package.json` for the sole `pyright` dependency.

6.2 Local installation

Base project installation follows a linear procedure:

1. clone the repository;
2. enter the project directory;
3. install required Python dependencies.

A possible command sequence is the following:

```
git clone https://github.com/AlexSantini10/distributed-sensor-hub.git
cd distributed-sensor-hub
pip install -r requirements.txt
```

At the end of the installation, the repository is ready both for direct execution of a single node and for running automated tests. A recommended initial check is to run `python -m pytest -m "not integration" -q`, so that local environment consistency can be verified immediately.

6.3 Configuration through environment variables

The entire node operational configuration is externalized through environment variables. This approach makes deployment easy to reproduce both locally and in containers, avoiding dependence on rigid configuration files and allowing different topologies to be described without code changes.

The `.env.example` file documents the main variables, including:

- node identification and binding: NODE_ID, HOST, PORT, WEB_API_PORT;
- initial bootstrap: BOOTSTRAP_PEERS;
- TCP server limits: SOCKET_TIMEOUT, ACCEPT_QUEUE_SIZE, MAX_CONNECTIONS, MAX_WORKERS;
- failure detector parameters: PHI_THRESHOLD_SUSPECT, PHI_THRESHOLD_DEAD, PHI_INITIAL_INTERVAL_
- push/pull replication cadence: GOSSIP_SYNC_INTERVAL_MS, GOSSIP_PUSH_*, GOSSIP_PULL_*;
- simulation of network delay and packet loss: NETWORK_DELAY_*, NETWORK_PACKET_LOSS_PROB;
- definition of local sensors: SENSORS and the SENSOR_<i>_* family.

The practical outcome is that the same software image can assume different roles simply by changing the variable set, a useful feature for both testing and classroom demonstrations.

6.4 Running a single node

For a minimal local run, it is sufficient to define a few essential variables and start `node.py`. A PowerShell example is:

```
$env:NODE_ID="node-1"
$env:HOST="0.0.0.0"
$env:PORT="9000"
$env:BOOTSTRAP_PEERS=""
$env:WEB_API_PORT="10000"
python node.py
```

The expected result is startup of the node with active TCP listener and HTTP server. In this mode the system is already queryable through web APIs, for example on `/api/state`, `/api/membership`, and `/api/introspection`.

6.5 Containerized deployment

Multi-node deployment is based on Docker Compose. The repository includes multiple Compose files, each targeting a different experimental scenario:

- `docker/docker-compose-base.yml`: minimal topology, quick to start, useful for smoke tests and quick demos;

- `docker/docker-compose-6-nodes.yml`: intermediate scenario, suitable for clearly observing state convergence and gossip dynamics;
- `docker/docker-compose-12-nodes.yml`: denser scenario, designed to stress dissemination and observability readability;
- Compose files dedicated to integration tests: used for deterministic scenarios and automated validation.

Startup is performed with commands such as:

```
docker compose -f docker/docker-compose-base.yml up --build -d
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
docker compose -f docker/docker-compose-12-nodes.yml up --build -d
```

The expected result is the creation of multiple homogeneous containers, each with:

- its own logical identity (`NODE_ID`);
- distinct TCP and HTTP ports exposed to the host;
- dedicated sets of simulated sensors;
- independently configured network and replication parameters;
- a shared log directory mounted as a volume for external inspection.

6.6 Engineering of Compose files

The `docker-compose` files do not merely instantiate identical node copies; they encode lab scenarios with heterogeneous profiles. In particular:

- bootstrap peers are distributed across nodes to avoid dependence on a single initial contact point;
- sensors differ by type, periodicity, latency, and measurement unit, so as to simulate non-uniform workloads;
- some nodes deliberately introduce higher delays, jitter, or packet-loss probabilities to make degradation and recovery phenomena observable;
- replicated-delta buffer size and push/pull thresholds are explicitly encoded in deployment, facilitating experimental tuning.

This choice is methodologically relevant: deployment is an integral part of the distributed experiment, not a mere operational detail. Compose files therefore function both as execution tools and as reproducible descriptions of scenarios analyzed during validation.

6.7 Startup verification

After deployment, verification can be conducted at three levels:

- infrastructure level, by checking container status with `docker compose ps`;
- application level, by querying HTTP endpoints on one or more nodes;
- system level, by running `python tests/integration/verify_cluster.py` to confirm startup, reachability, and baseline cluster availability.

In this way, deployment is not considered complete only when processes are active, but when the cluster effectively exhibits the expected distributed behavior.

6.8 Continuous delivery of the Docker image

To simplify node reuse outside the local development environment, the repository also includes a continuous-delivery pipeline based on GitHub Actions. On each push to the `main` branch, the workflow builds the node image from `docker/Dockerfile.base` and publishes it to GitHub Container Registry, keeping rolling tags such as `latest` and `main` up to date.

Creation of a GitHub Release instead occurs only when a version tag is associated with a commit, for example `v1.0.0`. In that case, the workflow also publishes the versioned image tag and generates a dedicated release containing the artifact digest and a small operational bundle: concise instructions, an example `.env` file, and a minimal `docker-compose` file already pinned to the released image. This separates the continuous-integration flow from publication of explicit versions intended for distribution.

This choice improves deployment traceability: the link between versioned source, built image, and operational instructions remains explicit, facilitating both demonstrations and reproduction of a specific state of the distributed system.

7 User Guide

This section describes operational system usage from the perspective of a technical user who wants to start the cluster, observe state convergence, and inspect introspection surfaces. Since the project does not expose external write capabilities, the main interaction consists of starting nodes, waiting for distributed-system evolution, and analyzing behavior through HTTP APIs and the web dashboard.

7.1 System startup

The most effective setup for demonstration is to start an already prepared Docker topology. The recommended scenario is the six-node one:

```
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
```

After startup, it is advisable to wait a few seconds so that nodes can complete bootstrap, peer-list exchange, initial heartbeats, and initial state replication. The expected effect is the progressive appearance of a shared membership view and sensor records replicated across nodes.

To stop the cluster:

```
docker compose -f docker/docker-compose-6-nodes.yml down
```

7.2 Direct API inspection

Each node exposes a read-only web API on the port defined by `WEB_API_PORT`. The most useful endpoints during operation are:

- `/api/state`: snapshot of replicated sensor state;
- `/api/updates`: stream or view of recent updates;
- `/api/membership`: local membership view;
- `/api/topology`: representation of known topology;
- `/api/introspection`: aggregated snapshot according to the `introspection/v1` contract;
- `/api/introspection/events`: recent control-plane events;
- `/api/introspection/metrics`: counters and indicators related to replication.

For example, in a base topology, `node-1` is normally reachable at `http://localhost:10000`. Querying the aggregated `/api/introspection` endpoint provides a useful view to verify:

- which peers are currently considered `alive`, `suspected`, or `dead`;
- which sensor records are known to the queried node;
- how many replication rounds and gossip messages have been observed;
- whether the delta buffer is sufficient or produces `DELTA_UNAVAILABLE`.

7.3 Using the web dashboard

For more convenient observation, a static dashboard is available in the `web/` directory. It requires no additional backend and consumes only the `/api/introspection` endpoint. To start it:

```
cd web
python -m http.server 8080
```

Then open `http://localhost:8080` in the browser and set the *API Base URL* to the HTTP endpoint of one node, for example `http://localhost:10000`.

The dashboard provides the main analysis views:

- graph of observed topology;
- peer status and related liveness semantics;
- table of replicated sensors with source and timestamp;
- timeline of recent events;
- synthetic metrics strip for gossip, full sync, and delta replication.

In a stable cluster, the expected outcome is a topologically coherent view, a majority of peers in `alive` state, and relatively recent sensor timestamps for high-frequency sources.

The repository does not include pre-captured screenshots, because the dashboard is a live view of the current experiment. The most useful screenshots for documenting a run are the topology graph and metrics strip, the membership/status panel, and the replicated sensor table after the six-node topology has been active for at least a short convergence interval. A complementary screenshot can be taken from the browser or terminal at `http://localhost:10000/api/introspection`, showing the raw introspection snapshot consumed by the dashboard.

7.4 Interpreting key observable signals

During operation, it is useful to interpret several operational indicators:

- if many `sensor_update_received` events appear with `applied:false`, the node is receiving updates already superseded by the LWW criterion;
- if the `get_delta_unavailable_total` counter increases, the incremental-delta buffer may be too short for current load or reconnection times;
- if a peer frequently transitions from `alive` to `suspected`, the simulated network or heartbeat interval may be introducing transients visible to the failure detector;
- if sensor timestamps appear too old in the UI, it is advisable to reduce pull pressure or increase `REPLICATION_DELTA_MAXLEN`.

The dashboard is therefore not only a demonstration tool, but also an important component for experimental diagnosis of distributed behavior.

7.5 Recommended usage scenarios

From a didactic and demonstration standpoint, the most significant scenarios are:

- **convergence under normal conditions**: start the cluster and observe progressive state alignment;
- **crash and recovery**: temporarily stop one node and verify catch-up through deltas or full sync;
- **degraded network**: use delays and packet loss configured in Compose to observe effects on heartbeats and data freshness;
- **observable scalability**: compare 2-node, 6-node, and 12-node topologies through the same UI.

For repeated fault-injection scenarios, the script `manual_tests/compose_chaos.py` is also available and useful to introduce controlled stop/restart cycles of individual services while the cluster is running.

7.6 Using an image published through release

If building the Docker image locally is not desired, it is possible to use a release generated by the continuous-delivery pipeline directly. However, a release is not created for every commit: it is produced when a version tag is assigned to a commit, for example `v1.0.0`, associated with the node image uploaded to GitHub Container Registry.

The operational flow is as follows:

1. create the version tag locally and publish it to the remote repository;
2. open the repository *Releases* section;
3. select the release corresponding to the desired commit;
4. download attached assets, especially the example `.env` file and minimal `docker-compose`;
5. adapt the main parameters of the `.env` file;
6. start the container through Compose or use the `docker run` command reported in release notes.

An example of release publication is:

```
git tag v1.0.0
git push origin v1.0.0
```

A typical execution example using attached assets is:

```
cp docker-node.env.example docker-node.env
docker compose -f docker-compose.release.yml up -d
```

Alternatively, direct startup by digest is also available:

```
docker pull ghcr.io/<owner>/<repo>@sha256:<digest>
docker run --rm -p 9000:9000 -p 10000:10000 \
  -e NODE_ID=node-1 \
  -e HOST=0.0.0.0 \
  -e PORT=9000 \
  -e WEB_API_PORT=10000 \
  -e BOOTSTRAP_PEERS= \
  ghcr.io/<owner>/<repo>@sha256:<digest>
```

This mode is useful when rapidly running a node consistent with a specific commit on the main branch, while preserving environment reproducibility through an immutable reference to the image digest.

8 Self-evaluation

This project was carried out as an individual project by Alex Santini. The following self-evaluation therefore reports the complete personal contribution to the design, implementation, validation, deployment, and documentation activities.

8.1 Alex Santini

Role in the project. As the only project member, I covered the full development lifecycle: requirements analysis, protocol and architecture design, runtime implementation, testing, containerized deployment, observability support, and final report writing. This included both the distributed core and the supporting artifacts needed to reproduce multi-node experiments.

Main contributions. The main contribution was the design and implementation of a decentralized peer-to-peer sensor-state replicator based on autonomous nodes, TCP communication, JSON protocol messages, push-pull gossip, and timestamp-based Last-Writer-Wins merging. I also implemented the membership and failure-observation mechanisms, including heartbeat-based liveness tracking and Phi Accrual failure suspicion, together with recovery paths based on incremental deltas and full-state synchronization.

On the operational side, I prepared Docker Compose topologies for reproducible multi-node simulations and configured sensor profiles, bootstrap information, and fault-related parameters through environment variables. I also developed the read-only HTTP and introspection surfaces used by tests, scripts, and the optional web dashboard to observe sensor values, membership, topology, events, and replication metrics.

Validation was another important part of the work. I developed and maintained tests for protocol handling, networking, state convergence, membership, failure detection, sensors, web APIs, Docker-based scenarios, crash/restart recovery, and partition reconciliation. These tests helped connect the intended distributed-systems properties with observable behavior in repeatable experiments.

Product strengths. The strongest aspect of the product is that the main distributed mechanisms are implemented explicitly rather than hidden behind a broker or external middleware. This makes gossip dissemination, membership propagation, failure suspicion, timestamp-based conflict resolution, and recovery behavior directly observable. The project also benefits from a reproducible deployment model, a clear separation between control plane, data plane, and observability plane, and a validation strategy that combines unit tests, integration tests, Docker scenarios, and manual observation.

Weaknesses and limitations. The system remains a course-project prototype rather than a production-ready smart-city platform. It does not implement authentication, authorization, or transport encryption; it stores runtime state in memory rather than in durable local storage; and its Last-Writer-Wins policy is deterministic but semantically simple. Larger topologies would also require more systematic quantitative evaluation of gossip overhead, convergence latency, and failure-detector sensitivity.

Personal evaluation of contribution. The most successful decisions were keeping the architecture decentralized, making observability a first-class part of the system, and validating recovery and convergence through reproducible Docker scenarios. The parts that required the most iteration were the balance between incremental anti-entropy and full synchronization, and the tuning of failure detection under delayed or lossy communication. With more time, I would strengthen the quantitative evaluation, add durable snapshots, and harden the security boundaries while preserving the educational transparency of the implementation.

Group members.

- Alex Santini – `alex.santini2@studio.unibo.it`

9 Future works

Although the project achieves its educational objectives in terms of distributed replication, eventual convergence, and observability, the implemented architecture leaves room for several significant extensions, both from an engineering and an experimental perspective.

9.1 Security and operational hardening

The first area for evolution concerns security. In the current version, the system operates in a controlled context and does not implement peer authentication, HTTP API protection, or channel encryption. A natural evolution would be to introduce:

- mutual authentication among nodes;
- protection of HTTP endpoints through tokens or mTLS;
- stricter validation of the sender's logical identity with respect to the transport channel;
- authorization policies to distinguish observational and administrative roles.

These changes would bring the system closer to a real-world scenario, where fault tolerance must coexist with baseline trust requirements and control-plane protection.

9.2 Persistence and historical data

At present, nodes retain only in-memory volatile state. This choice is consistent with the project's focus, but limits the system's ability to survive extended restarts or provide long-term temporal analysis. A relevant extension would therefore be the introduction of:

- local persistence of sensor records and recent deltas;
- periodic snapshots of membership and topology metadata;
- startup restore mechanisms to reduce bootstrap cost;
- separate historical archiving for offline analysis and comparison across experimental runs.

This evolution would also enable more rigorous comparisons of *cold-start* behavior, recovery times, and temporary information loss after extended crashes.

9.3 Richer replication semantics

The adopted Last-Writer-Wins criterion is simple, deterministic, and suitable for the project context, but it is also a simplification. Looking ahead, it would be interesting to evaluate:

- the use of data-type-specific CRDTs;
- differentiated merge policies by sensor category;
- version vectors or causal metadata to better distinguish true concurrency from simple delayed arrival;
- adaptive strategies between incremental replication and full sync based on observed divergence.

These extensions would enable deeper study of the trade-off between implementation simplicity, amount of exchanged metadata, and semantic quality of convergence.

9.4 Experiments on larger topologies

Another natural direction concerns extending experimental scale. Current topologies already allow observation of interesting phenomena, but extending to larger clusters or more heterogeneous connectivity would make it possible to:

- evaluate the impact of topology density on gossip traffic;
- study failure-detector sensitivity in less regular networks;
- compare fan-out, sync intervals, and delta-buffer sizes under higher loads;
- verify readability and sustainability of observability as node count grows.

From an experimental standpoint, it would also be useful to complement this with systematic quantitative metrics on convergence latency, traffic volume, and recovery costs.

9.5 Advanced observability and analysis tooling

The project already provides a solid read-only introspection baseline, but it could be further extended through:

- export of metrics to standard monitoring stacks;
- automatic alerting on critical stale-data or `DELTA_UNAVAILABLE` thresholds;
- persistent temporal tracing of control-plane events;
- automated comparison across different replication and failure-detection configurations.

In this way, the system would become not only a functional prototype, but also a more mature experimental platform for systematically analyzing the consequences of architectural choices.